Lab 1 Reflection — Adapter Pattern

Riejed Aniko L. Macatula | BSIT 3A | Integrative Programming and Technologies

The smell

ReportExporterWithoutAdapter handles CSV, JSON, and text export inside one method with an if/elseif chain, one branch per format. Each branch calls a different legacy library in a different way: fputcsv for CSV, json_encode for JSON, manual fwrite loops for text. The class has to know about every writer that exists, and so does every caller that constructs it with a $format string. Adding a fourth format means opening this file and adding another elseif. That's the Open-Closed Principle breaking in real time: every new format forces a change to code that already works.

The fix

Adapter splits this into four pieces. FileWriter is the target interface, with a single write($filename, $rows): bool method that ReportExporter depends on. The three legacy libraries (CsvLibrary, JsonLibrary, TextLibrary) stay untouched as adaptees. Each adapter, CsvFileAdapter, JsonFileAdapter, TextFileAdapter, wraps one library and translates its mismatched method into write(). The translation shows up clearest in TextFileAdapter::write(): TextLibrary::appendLine() only writes one line at a time, so the adapter loops over $rows, builds each line, and calls appendLine() once per row, returning false the moment a write fails. ReportExporter only ever calls $this->writer->write(...) and has no idea which library sits behind that call. I proved this by adding a fourth adapter, XmlWriterAdapter, around a new XmlLibrary class. That was the only new class needed; ReportExporter didn't change, and the existing csv/json/text paths kept working as before.

Gamma citation

Gamma et al. (1994, p. 139) define Adapter as a pattern that converts "the interface of a class into another interface clients expect." FileWriter is the interface the client expects, and each adapter converts one legacy library's interface into it without touching the library or the client.